# Linux网络--IO多路转接层03 epoll事件模型与内核原理

> ⓘ 系列导航
>
> 上一篇：Linux网络--IO多路转接层02 poll原理与select服务器改写
> 系列索引：Linux网络--IO多路转接与Reactor系列总览
> 下一篇：Linux网络--IO多路转接层04 LT与ET及epoll服务器实现

## epoll 事件模型与内核原理

### 1. epoll 改变了什么

select 和 poll 的共同结构是：

```
应用每轮提交一整套关注对象
        -> 内核检查这套对象
        -> 应用从返回结果中再次筛选
```

epoll 把“长期关心什么”和“本次什么就绪”拆成两个阶段：

```
epoll_ctl：把 fd 和事件持久注册到内核
epoll_wait：等待并取走当前就绪的事件子集
```

因此，应用不必每轮重新提交全部 fd。一个连接加入 epoll 后可以长期存在，只有关心的事件发生变化时才调用 `epoll_ctl` 修改。

epoll 优化的重点是“等待多个 fd 时的管理成本”。它没有替代 `recv/send`，也不会自动完成协议解析、业务计算和数据拷贝。

### 2. 三个核心系统调用

2.1 创建 epoll 实例

现代代码优先使用：

```c
#include <sys/epoll.h>

int epoll_fd = epoll_create1(EPOLL_CLOEXEC);
if (epoll_fd == -1) {
    perror("epoll_create1");
}
```

返回的 `epoll_fd` 也是一个文件描述符。内核为这个 epoll 实例维护关注集合和就绪队列。

旧接口：

```c
int epoll_create(int size);
```

从 Linux 2.6.8 开始，`size` 不再表示容量，只要求传入正数。它不能限制 epoll 能管理的 fd 数量，也不表示红黑树或就绪队列的上限。

`EPOLL_CLOEXEC` 让 epoll fd 在 `exec` 成功后自动关闭，避免被子进程意外继承。

### 2.2 注册和修改事件

```c
int epoll_ctl(
    int epfd,
    int op,
    int fd,
    struct epoll_event *event
);
```

三个操作：

- `EPOLL_CTL_ADD`：添加新的 fd 和关心事件；
- `EPOLL_CTL_MOD`：修改已有 fd 的关心事件；
- `EPOLL_CTL_DEL`：删除 fd 的注册关系。

添加读事件：

```c
struct epoll_event ev = {0};
ev.events = EPOLLIN;
ev.data.fd = client_fd;
```

```
if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, client_fd, &ev) == -1) {
    perror("epoll_ctl ADD");
}
```

修改为同时关心读写：

```
ev.events = EPOLLIN | EPOLLOUT;
epoll_ctl(epoll_fd, EPOLL_CTL_MOD, client_fd, &ev);
```

删除时不再需要事件描述：

```
epoll_ctl(epoll_fd, EPOLL_CTL_DEL, client_fd, NULL);
```

`ADD` 和 `MOD` 传的是“用户希望内核关心哪些事件”，`DEL` 只关心要删除哪个 fd。

### 2.3 等待就绪事件

```
int epoll_wait(
    int epfd,
    struct epoll_event *events,
    int maxevents,
    int timeout
);
```

参数含义：

- `epfd`：epoll 实例 fd；
- `events`：用户分配的输出数组；
- `maxevents`：一次最多返回多少个事件；
- `timeout`：毫秒；`-1` 永久阻塞，`0` 立即返回，正数表示超时上限。

返回值：

- `> 0`：就绪事件数量；
- `= 0`：超时；
- `< 0`：错误。

返回值不是“数值最大的下标”，而是有效元素个数。只有 `events[0]` 到 `events[n - 1]` 有效：

```
epoll_event events[1024];
int n = epoll_wait(epoll_fd, events, 1024, -1);

for (int i = 0; i < n; ++i) {
    handle_event(events[i]);
}
```

如果就绪事件超过 `maxevents`，剩余项仍由 epoll 保存，可以在下一次等待中继续取得。

### 3. epoll_event 和用户数据

```
typedef union epoll_data {
    void    *ptr;
    int      fd;
    uint32_t u32;
    uint64_t u64;
} epoll_data_t;

struct epoll_event {
    uint32_t     events;
    epoll_data_t data;
};
```

`events` 是事件位图。`data` 属于用户，内核不会修改它的内容，只在事件返回时原样带回。

#### 3.1 保存 fd

```
ev.data.fd = fd;
```

事件返回后用 `events[i].data.fd` 找到 fd。

#### 3.2 保存对象指针

```
ev.data.ptr = connection;
```

返回后可以直接得到 `Connection*`。Reactor 常用这种设计，因为一次事件通常需要访问连接的缓冲区、回调、状态和所属 EventLoop，只拿到 fd 往往还要再查一次哈希表。

保存指针后，生命周期要求更严格：对象真正销毁前，必须先从 epoll 中删除相关 fd，并确保不会有排队中的事件继续引用该对象。

### 4. epoll 常用事件

| 事件 | 含义 |
| --- | --- |
| `EPOLLIN` | 可读；对端关闭时通常也可读，随后 `recv` 返回 0 |
| `EPOLLOUT` | 可写 |
| `EPOLLRDHUP` | 对端关闭写方向 |
| `EPOLLERR` | fd 发生错误 |
| `EPOLLHUP` | fd 挂断 |
| `EPOLLET` | 使用边缘触发 |
| `EPOLLONESHOT` | 本次触发后失活，需要 `EPOLL_CTL_MOD` 重新启用 |
| `EPOLLEXCLUSIVE` | 多个 epoll 实例等待同一 fd 时减少无效唤醒 |

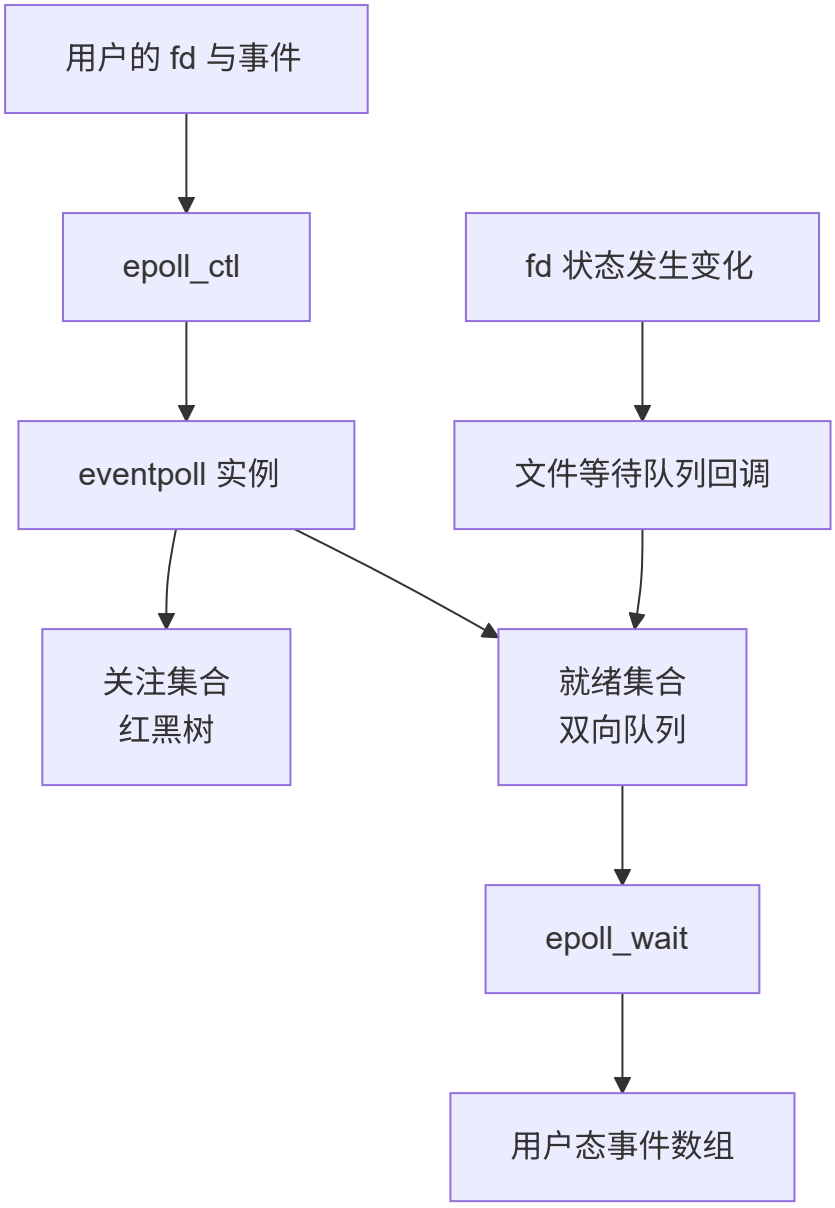
应用程序通常只显式注册 `EPOLLIN` 和需要的 `EPOLLOUT`，但 `EPOLLERR`、`EPOLLHUP` 仍可能出现，必须处理。

```
void Dispatch(uint32_t events) {
    if (events & (EPOLLERR | EPOLLHUP)) {
        HandleErrorOrHangup();
        return;
    }
    if (events & EPOLLIN) {
        HandleReadable();
    }
    if (events & EPOLLOUT) {
        HandleWritable();
    }
}
```

错误检查应放在读写之前。一个已经挂断的连接即使同时报告可读，也不应该继续按正常请求处理。

**5. 内核中的关注集合与就绪集合**

可以用下面的结构理解 epoll：




5.1 eventpoll

一个 epoll 实例对应内核中的 `eventpoll` 对象。与使用方式最相关的两个组织是：

- 保存所有已注册 fd 的红黑树；
- 保存当前就绪项的队列。

5.2 epitem

每个注册到 epoll 的 fd 都有对应的 `epitem`。其中会关联：

- 红黑树节点；
- 就绪链表的连接节点；
- 对应文件和 fd 的信息；
- 用户关心的事件；
- 指回所属 `eventpoll` 的指针。

红黑树适合反复添加、修改和删除，因为 fd 是唯一的整数键。一般认为这些维护操作是 `O(log N)`。

### 5.3 就绪队列

当 fd 的状态满足关心事件时，对应的 `epitem` 被加入就绪队列。`epoll_wait` 不必扫描红黑树中的全部节点，只需检查队列并取出就绪项。

就绪项返回后，是否继续留在队列中取决于触发模式：

- LT 下，只要条件仍成立，下一轮仍会被报告；
- ET 下，只有出现新的状态边沿时才重新通知。

## 6. 回调机制

select 和 poll 的等待路径需要让内核检查一组 fd。epoll 在注册时建立回调关系，状态变化会主动推动就绪项进入队列。

可以概括为：

```
epoll_ctl ADD
    -> 建立 epitem
    -> 把 epoll 回调关联到目标的等待队列

网络数据到达 / 发送缓冲区出现空间 / 连接状态变化
    -> 文件或协议栈唤醒等待者
    -> 执行 epoll 的回调
    -> 把 epitem 放入就绪队列

epoll_wait
    -> 检查并取走就绪项
    -> 把用户关心的事件复制到输出数组
```

这里的“回调”首先发生在内核对象之间，不是应用层注册的业务函数。应用看到的结果只是 `epoll_wait` 返回了某些 `epoll_event`。

## 7. epoll 的使用三部曲

### 7.1 创建实例

```
int epfd = epoll_create1(EPOLL_CLOEXEC);
```

### 7.2 注册监听 socket

服务器启动时还没有客户端连接，但必须让内核在监听 socket 出现新连接时唤醒事件循环：

```
epoll_event ev = {0};
ev.events = EPOLLIN;
ev.data.fd = listen_fd;
epoll_ctl(epfd, EPOLL_CTL_ADD, listen_fd, &ev);
```

如果不注册任何 fd，`epoll_wait` 没有可等待对象，服务器就不会知道何时应该推进事件。

### 7.3 进入事件循环

```
for (;;) {
    epoll_event events[1024];
    int n = epoll_wait(epfd, events, 1024, -1);

    if (n < 0) {
        if (errno == EINTR) continue;
        break;
    }

    for (int i = 0; i < n; ++i) {
        int fd = events[i].data.fd;

        if (fd == listen_fd) {
            // accept
        } else {
            // 处理连接事件
        }
```

```
    }
}
```

关闭 epoll 实例时调用 `close(epfd)`。epoll fd 是普通文件描述符，但其管理关系仍需按连接生命周期正确维护。

## 8. 从 poll 到 epoll 的完整路径

### 8.1 先看 poll 的边界

poll 由用户传入数组，内核必须检查数组元素。返回后用户仍要从数组中寻找 `revents`。

### 8.2 再看 epoll_ctl

`epoll_ctl ADD` 不是“发起一次等待”，而是“修改内核长期保存的关注集合”。真正等待发生在 `epoll_wait`。

### 8.3 接着看红黑树

红黑树中保存的是“以后需要持续关心什么”。fd 是键，所以添加、删除和修改可以针对单个 fd 完成。

### 8.4 然后看就绪队列

红黑树节点并不需要从树上搬到另一棵树。注册节点可以额外挂入就绪链表的连接结构，因此“关注”和“就绪”是两个重叠但不冲突的状态。

### 8.5 最后看 epoll_wait

等待路径关注就绪队列，而不是遍历红黑树。返回输出数组后，用户只遍历 `[0, n)`。

## 9. epoll 的复杂度应该怎么说

“epoll_wait 是 O(1)”只适合描述某一层操作：有无数个 fd 都注册着，检测当前是否就绪时不随全部 fd 数线性增长。

更完整的描述是：

- 注册、修改、删除关注项：通常与红黑树维护有关，可写为 `O(log N)`；
- 等待就绪：不需要扫描全部 `N` 个注册 fd；
- 返回并处理 `R` 个就绪事件：成本与应用处理这 `R` 个事件的成本有关，可写为 `O(R)`；
- 真正的数据读写、协议解析、业务计算仍由应用承担。

因此，不能把整个网络服务器的端到端复杂度宣布为 `O(1)`。

### 10. 数据拷贝与 mmap 误区

epoll 减少了重复注册和全量扫描，但不是“完全没有拷贝”。

用户调用 `epoll_ctl` 时，事件描述需要从用户态传给内核；`epoll_wait` 时，内核把本次就绪的事件数组复制到用户提供的 `epoll_event` 缓冲区。

有一种常见说法是：

> epoll 用 mmap 把内核就绪队列直接映射到用户态，所以完全没有拷贝。

这个说法不准确。应用提供的 `epoll_event` 数组仍在用户空间，`epoll_wait` 仍需要把就绪描述写回该数组。epoll 的主要优化来自注册与就绪集合分离，而不是零拷贝传输业务数据。

### 11. 三种多路转接的横向比较

| 维度 | select | poll | epoll |
| --- | --- | --- | --- |
| 关注集合 | `fd_set` 位图 | `pollfd` 数组 | 内核持久注册 |
| 输入/输出 | 同一位图先输入后输出 | `events` / `revents` 分离 | 注册事件 / 本次就绪事件分离 |
| 每轮提交 | 全部集合 | 全部数组 | 不重新提交未变化项 |
| 等待成本 | 复制并检查集合 | 复制并检查数组 | 检查就绪队列 |
| 返回后处理 | 扫描范围 | 扫描整个数组 | 只处理返回的 `n` 项 |
| 数量限制 | 常见 `FD_SETSIZE` 位图限制 | 受资源和进程上限约束 | 受资源和进程上限约束 |
| 触发方式 | 类 LT | 类 LT | LT / ET |
| 主要优势 | 简单、兼容性好 | 事件表达清楚 | 大量连接、少量活跃 |

epoll 并非在所有场景都一定比 select 或 poll 快。连接很少、每个连接都频繁活跃时，epoll 的注册和管理成本未必低于一次简单等待。工程上应根据连接规模、活跃比例和实际测量决定。

### 12. epoll 适合什么场景

epoll 特别适合：

- 同时维护大量长连接；
- 任意时刻只有较少连接真正活跃；
- 服务器需要频繁增加、删除和修改连接；
- 应用愿意采用事件循环，而不是为每个连接阻塞一个线程。

它不适合用来掩盖以下问题：

- 回调中执行慢速业务；
- 每个请求都进行大量同步磁盘 I/O；
- 无法正确处理半包、粘包和慢发送；
- 多个线程同时操作同一个连接状态；
- 连接数量超过系统资源限制。

### 13. 从“知道 epoll”到“写对 epoll 服务器”

到这里，我们已经知道 epoll 为什么不必全量扫描，以及三类接口怎样协作。但工程中的难点才刚刚开始：

- 一个 socket 可读时，应该读几次；
- 一个监听 socket 可读时，要 accept 几次；
- 发送缓冲区满了怎么办；
- LT 和 ET 分别怎样通知；
- 什么时候打开和关闭 `EPOLLOUT`；
- 连接关闭和对象销毁按什么顺序执行。

下一篇专门处理这些问题，先讲 LT/ET 的完整语义，再分别实现 LT 与 ET 版本的 epoll 服务器。